

Fundamentals

Socket Exhaustion

Preparation

1. Make sure WeatherApi is running:
 - Open terminal
 - Navigate to WeatherApi csproj folder
 - Run `dotnet run --launch-profile https`
 - Check if `https://localhost:<port>/weatherforecast` return valid response
2. Make sure project is unloaded in VS, otherwise it will try to build but will fail because the api is running

Demo

- Make sure that Common Language Runtime Exceptions are enabled
- Show the code of the demo
- Run it, somewhere between 2000 and 3000 calls errors should appear
- Move httpClient instantiation outside the loop and reuse instance
- Now it won't crash

ServiceProvider in simple apps

- Comment the socket exhaustion region
- Uncomment the SP region
- Run the naive implementation

Add ServiceCollection

- Add `Microsoft.Extensions.DependencyInjection` and `Microsoft.Extensions.Http` NuGet packages
- Add `var services = new ServiceCollection();` in Program.cs
- Next we can register the httpClientfactory `services.AddHttpClient();`

Use IHttpHttpClientFactory

- build the service provider `var provider = services.BuildServiceProvider();`
- Pull the factory out of the DI container `var factory = provider.GetRequiredService<IHttpClientFactory>();`
- Create the HttpClient `var client = factory.CreateClient();`
- Run the app, we are now using single instance through dependency injection

IHttpClientFactory usage

- Run the blazor app, show the weather page
- Go to the code, notice it uses `GetStringAsync`
- Start refactor to best practices

Best practices for calls

- Replace `GetStringAsync` line with:

```
HttpResponseMessage response = await
client.GetAsync("https://localhost:7060/weatherforecasts");
response.EnsureSuccessStatusCode();

string json = await response.Content.ReadAsStringAsync();
```

Add Try Catch

```
catch (HttpRequestException ex)
{
    Console.WriteLine($"HTTP Error: {ex.Message}");
    _forecasts = [];
}
```

Also catch `TaskCanceledException` (getasync throws `OperationCanceledException`, but taskcancel is subclass of that)

```
catch (TaskCanceledException ex)
{
    // Handle timeout
    Console.WriteLine($"Request timed out: {ex.Message}");
    _forecasts = [];
}
```

And unhandled exceptions

```
catch (Exception ex)
{
    // Handle other unexpected errors
    Console.WriteLine($"Unexpected error: {ex.Message}");
    _forecasts = [];
}
```

Named Clients

- change the `CreateClient` to the named client overload

```
var client = Factory.CreateClient("weatherapi");
```

- remove the baseaddress from the `GetAsync` call (Make sure to call plural weatherforecastS)

```
HttpResponseMessage response = await client.GetAsync("/weatherforecasts");
```

- in `Program.cs` change the registration

```
builder.Services.AddHttpClient("weatherapi", client =>
{
    client.BaseAddress = new Uri("https://localhost:7060");
});
```

Speaker Note: "Named clients let you centralize configuration for each service you call, instead of scattering headers and base addresses throughout your code."

Typed Clients

- Create `WeatherClient` class, inject `HttpClient`

```
public class WeatherClient(HttpClient client)
```

- Copy the `Forecast` class into it and make it public
- Create a `GetWeather` method and copy the fetch code from `Weather.razor`
- In `Weather.razor` inject `WeatherClient`

```
[Inject]
public required WeatherClient Client { get; set; }
```

- Use the client to fetch the forecasts

```
protected override async Task OnInitializedAsync()
{
    _forecasts = await Client.GetWeatherAsync();
}
```

- Remove the name from the `HttpClient` registration and add the type in `Program.cs`

```
builder.Services.AddHttpClient<WeatherClient>(client =>
{
    client.BaseAddress = new Uri("https://localhost:7060");
});
```

Speaker Note: "Typed clients encapsulate configuration and usage. Your controllers or services just depend on `GitHubClient`, not raw `HttpClient`."

Handlers

Creating a custom handler

- Add a **Handlers** folder to the frontend
- Create a **LoggingHandler** class and inherit from **DelegatingHandler**
- Override the **SendAsync** method
- Add **Console.WriteLine(\$"==== HANDLER: Logging request to: {request.RequestUri}");**
- In Program.cs register the handler as scoped
- add it to **AddHttpClient .AddHttpMessageHandler<LoggingHandler>();**
- Run the app, show the log output in the console (not browser console, app output)

Chaining handlers

- Copy the **LoggingHandler** and change the name to **AuthHandler**
- Add a dummy Bearer token to the headers

```
Console.WriteLine("==== HANDLER: Adding auth header");  
request.Headers.Authorization = new AuthenticationHeaderValue("Bearer",  
"MyToken");
```

- register the header as scoped, and put it as the first handler in the httpClient
- Set breakpoint in LoggingHandler and run app
- Show that the header is filled in by the time the request reaches that handler
- Show order of logging statements in the console => same as order of registration

Handle responses

- Change the handlers to await the SendAsync and log something in both

```
var response = await base.SendAsync(request, cancellationToken);  
Console.WriteLine($"=== Response received: HTTP {response.StatusCode}");  
  
return response;
```

- Run the app, notice responses go in reverse order, which matches the diagram in the slides

Adding Otel

- Add NuGet packages:

```
OpenTelemetry.Instrumentation.Http  
OpenTelemetry.Exporter.Console
```

```
using var meterProvider = Sdk.CreateMeterProviderBuilder()  
    .AddHttpClientInstrumentation()  
    .AddConsoleExporter()  
    .Build();
```

- Run the app, show the telemetry in the console
- Add Options to the **AddHttpClientInstrumentation**

```
.AddHttpClientInstrumentation(options =>  
{  
    options.EnrichWithHttpRequestMessage = (activity, message) =>  
    {  
        activity.SetTag("version", "1.0 beta");  
    };  
})
```

- Run again, notice the tag appearing in the console output

Resiliency

Polly Retry

- Show retry-demo page
- Show that we need to click the button multiple times until success
- Add the following nuget package:

```
Microsoft.Extensions.Http.Polly
```

- In Program.cs, define the policy:

```
var retryPolicy = HttpClientExtensions.HandleTransientHttpError()  
    .RetryAsync(3);
```

- Apply it to the httpclient

```
builder.Services.AddHttpClient("weatherApi", client =>  
{  
    client.BaseAddress = new Uri("https://localhost:7060");  
    client.Timeout = TimeSpan.FromSeconds(30);  
})  
.AddHttpMessageHandler<LoggingHandler>()  
.AddPolicyHandler(retryPolicy);
```

- Show app again, now it retries
- Show logging in the console logs
- Change the policy from `RetryAsync` to

```
.WaitAndRetryAsync(3, retryAttempt => TimeSpan.FromSeconds(Math.Pow(2, retryAttempt)));
```

- Show logs again and notice that it waits longer between each try

Circuit Breaker

- Replace `WaitAndRetryAsync` with `.GetCircuitBreakerPolicy();`
- Start the application
- Hit the red button three times, notice the circuit message in console
- Click another button => error that circuit is open
- Wait 5 seconds
- Notice that it works again
- Show the code in `.GetCircuitBreakerPolicy();`